BCSE307P COMPILER DESIGN LABORATORY • REVIEW 3 FINAL

QueryLang: A Compiler-Style Query Engine with Rule-Based Query Optimization

Design, Formal Verification, and Empirical Benchmarking of a Domain-Specific Relational Compiler

STUDENT CANDIDATE
Aman Katakwar (24BCE0160)

FACULTY GUIDE
Prof. Ranjithkumar S

SCHOOL / INSTITUTION
SCOPE, VIT Vellore

EVALUATION PHASE
Review 3 / Final Viva

1. Executive Summary & Vision

Demonstrating genuine compiler construction paradigms on declarative query languages

Industrial Rationale: DBMS as Compilers

Contemporary relational engines (SQLite, DuckDB, PostgreSQL) are **domain-specific compilers**. They do not interpret raw query strings; they parse, semantically validate, translate to Intermediate Representation (IR), optimize algebraically, and walk physical operator plans.

- **Declarative Abstraction:** Queries specify *what* data is desired, leaving the compiler to construct the *optimal execution strategy*.
- **Compilation Rigor:** Lexical scanning, LL(1) recursive descent, typed ASTs, symbol table catalog binding, and algebraic rewrites.

Scope of Review 3 Delivery

All planned Phase 3 milestones fully realized from first principles using standard Python 3.12 (Zero third-party dependencies):

- Rule-based Query Optimizer (Constant Folding, Predicate Simplification, Filter Pushdown).
- Multi-table relational `INNER JOIN` with qualified attributes.
- Scalar Aggregations (`COUNT` , `AVG` , `SUM` , `MIN` , `MAX`).

Key Engineering Achievements

Metric / Phase	Status / Verified Achievement
Grammar Precedence	6-tier unambiguous hierarchy (Boolean < Comparison < Arithmetic)
Optimizer Rules	Constant Folding, Tautology Simplification, Join Filter Pushdown
Benchmark Speedup	**1.89x to 2.28x speedup** on 100, 1k, and 10k synthetic datasets
Test Matrix	**25 / 25 formal tests PASS (100%)** + 26 PyUnit unit tests
Visualizer	Graphviz DOT export for AST and Relational Plans

2. Problem Statement & Motivation

Overcoming naive interpreter bottlenecks through formal compiler pipeline design

The Naive Interpreter Pitfall

- **Regex / String Splitting:** Brittle, cannot handle nested parentheses, multi-level operator precedence, or string escapes.
- **Late Semantic Binding:** Errors (unknown columns, type mismatches) surface mid-execution rather than at compile time.
- **Cartesian Join Explosions:** Joins without filter pushdown execute full cartesian products: \(O(N imes M)\) comparisons.

The QueryLang Solution

A decoupled 6-phase compiler pipeline guaranteeing:

- **Deterministic Parsing:** Clean LL(1) grammar without left recursion.
- **Compile-Time Safety:** Static type checking and catalog binding before data evaluation.
- **Algebraic Minimization:** Filter pushdown reduces join inputs from \(N imes M\) to \(\sigma(N) imes M\).

Execution Complexity Contrast

```
-- Query with Join & Filter:
SELECT s.name, d.dept_name FROM students s
JOIN departments d ON s.branch = d.dept_name
WHERE s.age > 20;

[NAIVE / UNOPTIMIZED EXECUTION]
Step 1: Nested Loop Join: N * M iterations = 10,000 * 5 = 50,000
comparisons
Step 2: Filter applied after join over all 50,000 candidate
records.

[QUERYLANG OPTIMIZED EXECUTION (FILTER PUSHDOWN)]
Step 1: Filter pushed to SCAN[students]: 10,000 * 0.267 = 2,672
rows
Step 2: Join executed on filtered subset: 2,672 * 5 = 13,360
comparisons
RESULT: 73.3% reduction in join comparisons!
```

3. QueryLang DSL & Extended Language Specification

Declarative SQL dialect supporting relational selections, joins, arithmetic, and aggregates

Core Language Features

- **Projections:** Column lists (`name, cgpa`), wildcard (`*`), and qualified identifiers (`students.name`).
- **Multi-Table Joins:** Explicit `[INNER] JOIN ... ON <condition>` syntax.
- **Aggregations:** `COUNT(*)`, `COUNT(col)`, `AVG`, `SUM`, `MIN`, `MAX`.
- **Arithmetic Expressions:** Full arithmetic support (`+`, `-`, `*`, `/`, `( )`) in conditions.
- **Ordering & Limits:** `ORDER BY <col> [ASC|DESC]` and `LIMIT <int>`.

Representative Canonical Queries

```
-- 1. Selection, Projection, Ordering, Limit
SELECT name, branch, cgpa FROM students
WHERE age >= 21 ORDER BY cgpa DESC LIMIT 3;

-- 2. Multi-Table INNER JOIN with Qualified Attributes
SELECT students.name, departments.dept_name, departments.building
FROM students
JOIN departments ON students.branch = departments.dept_name
WHERE students.cgpa >= 8.5;

-- 3. Optimization Idiom (Constant Folding + Filter Pushdown)
SELECT students.name, departments.dept_name
FROM students
JOIN departments ON students.branch = departments.dept_name
WHERE students.age > (18 + 2) AND (1 = 1);

-- 4. Statistical Aggregations
SELECT COUNT(*), AVG(cgpa), MAX(cgpa), MIN(age) FROM students;
```

4. Formal EBNF Grammar & Precedence Hierarchy

Strict, unambiguous grammar resolving boolean, relational, and arithmetic precedence

Operator Precedence Hierarchy

Level	Operators	Associativity
1 (Lowest)	`OR`	Left-to-Right
2	`AND`	Left-to-Right
3	`=`, `!=`, `>`, `<`, `>=`, `<=`	Non-associative
4	`+`, `-`	Left-to-Right
5	`*`, `/`	Left-to-Right
6 (Highest)	`( ... )`, Identifiers, Literals	Unary / Atom

Grammar Specification (EBNF Snippet)

```
Query           ::= "SELECT" Projection "FROM" TableReference [
JoinClause ]
                    [ "WHERE" Expression ]
                    [ "ORDER" "BY" QualifiedColumn [ "ASC" |
"DESC" ] ]
                    [ "LIMIT" INTEGER ] ";"

Projection      ::= "*" | ProjectItem ( "," ProjectItem )*
ProjectItem     ::= AggregateFunction | QualifiedColumn
AggregateFunction::= ( "COUNT" "(" ( "*" | QualifiedColumn ) ")" )
                   | ( ( "SUM" | "AVG" | "MIN" | "MAX" ) "("
QualifiedColumn ")" )

Expression      ::= LogicalOrExpr
LogicalOrExpr   ::= LogicalAndExpr ( "OR" LogicalAndExpr )*
LogicalAndExpr  ::= ComparisonExpr ( "AND" ComparisonExpr )*
ComparisonExpr  ::= AdditiveExpr [ ( "=" | "!=" | ">" | "<" | ">="
| "<=" ) AdditiveExpr ]
AdditiveExpr    ::= MultiplicativeExpr ( ( "+" | "-" )
MultiplicativeExpr )*
MultiplicativeExpr::= PrimaryExpr ( ( "*" | "/" ) PrimaryExpr )*
```

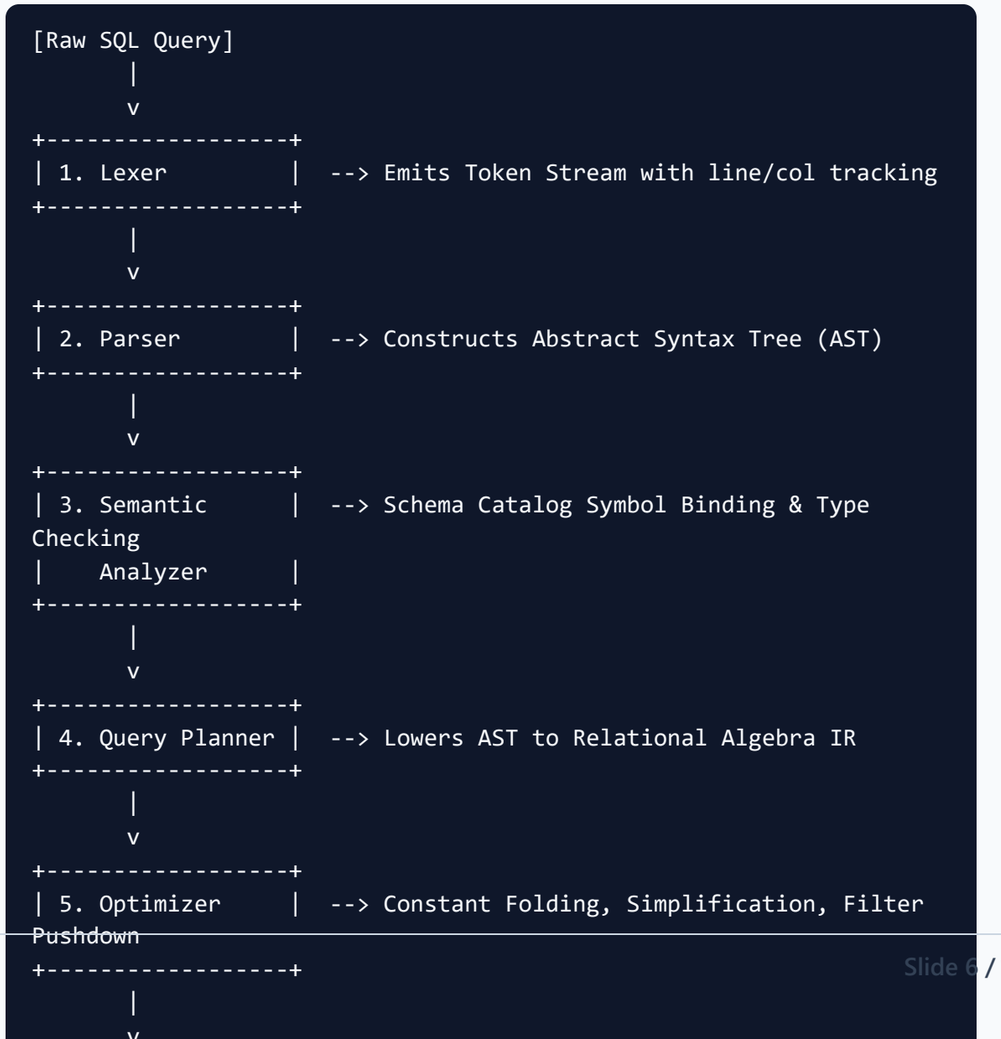
5. End-to-End Compiler Pipeline Architecture

Modular 6-stage compiler lowering queries from high-level text to optimized execution

Pipeline Stages & Module Responsibilities

- **Stage 1: Lexical Analysis (`lexer/`):** Converts raw query text into token stream with coordinates.
- **Stage 2: Syntax Analysis (`parser/`):** LL(1) recursive descent builds typed AST.
- **Stage 3: Semantic Analysis (`semantic/`):** Multi-schema catalog symbol binding & type inference.
- **Stage 4: Query Planner (`planner/`):** Emits Relational Algebra Logical Plan (IR).
- **Stage 5: Query Optimizer (`optimizer/`):** Constant folding, simplification & filter pushdown.
- **Stage 6: Execution Engine (`execution/`):** Evaluates optimized plan over in-memory relation data.

Compiler Architecture Diagram



6. Front-End: Lexical Analysis & Token Scanning

Linear-time scanning with 27 token types and coordinate-aware diagnostic recovery

Scanner Architecture (`lexer/lexer.py`)

- **Single Linear Pass:** Master regex compiled with `re.VERBOSE` and `re.IGNORECASE`.
- **Token Representation:** Each `Token` records `TokenType`, literal value, source line, and column position.
- **Review 3 Token Extensions:**
  - Keywords: `JOIN`, `INNER`, `ON`, `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.
  - Operators: `+`, `-`, `*`, `/`, `.` (DOT for qualified attributes).

Lexical Error Diagnostic Output

```
[LEXICAL ERROR] at Line 1, Column 13: Invalid character sequence
'@'
  --> SELECT name @ students;
                  ^
  HINT: Remove or replace the unsupported character.

Token Stream Sample:
  Token(SELECT, line=1, col=1)
  Token(IDENTIFIER, 'name', line=1, col=8)
  Token(COMMA, line=1, col=12)
  Token(IDENTIFIER, 'branch', line=1, col=14)
  Token(FROM, line=1, col=21)
  Token(IDENTIFIER, 'students', line=1, col=26)
  Token(SEMICOLON, line=1, col=34)
  Token(EOF, line=1, col=35)
```

7. Front-End: Syntax Analysis & AST Construction

LL(1) recursive-descent parser producing strongly typed, hierarchical syntax trees

Parser Architecture (`parser/parser.py`)

- **Deterministic LL(1):** Single-token lookahead without backtracking overhead (\(O(N)\) parsing time).
- **AST Node Hierarchy (`ast/nodes.py`):**
  - `SelectQueryNode` : Complete query representation.
  - `TableNode` & `JoinNode` : Source relations and joins.
  - `ColumnNode` : Qualified / unqualified column refs.
  - `AggregateNode` : Statistical aggregate expressions.
  - `ArithmeticOpNode` & `BinaryOpNode` : Expression trees.

Generated AST Tree Representation

```
SelectQueryNode
|-- Projected Columns: [name, branch, cgpa]
|-- Source Table: students
|-- WHERE Condition:
|   \-- BinaryOpNode(AND)
|       |-- BinaryOpNode(>=)
|       |   |-- ColumnNode(age)
|       |   \-- ArithmeticOpNode(+)
|       |       |-- LiteralNode(18 : INTEGER)
|       |       \-- LiteralNode(2 : INTEGER)
|       \-- BinaryOpNode(=)
|           |-- LiteralNode(1 : INTEGER)
|           \-- LiteralNode(1 : INTEGER)
|-- ORDER BY: cgpa DESC
\-- LIMIT: 3
```

8. Semantic Analysis & Multi-Schema Symbol Binding

Cross-schema catalog validation, column disambiguation, and static type checking

Schema Catalog as Symbol Table

Maintains table symbols and column symbol dictionaries across all active relations:

- `students` (id:INT, name:STR, age:INT, branch:STR, cgpa:FLOAT)
- `departments` (dept_id:INT, dept_name:STR, hod:STR, building:STR)

Static Type Checking Rules

- **Comparisons:** Rejects incompatible operands (e.g., `STRING > INTEGER`).
- **Arithmetic:** Enforces numeric operands (`INTEGER` or `FLOAT`).
- **Aggregates:** Restricts `SUM` and `AVG` strictly to numeric columns.

Semantic Diagnostic Examples

```
-- Unknown Column Error:
[SEMANTIC ERROR] at Line 1, Column 8:
Column 'salary' does not exist in table 'students'.
  --> SELECT salary FROM students;
            ^
  HINT: Available columns: id, name, age, branch, cgpa

------------------------------------------------
-- Static Type Mismatch Error:
[SEMANTIC ERROR] at Line 1, Column 38:
Operator '>' cannot be applied to STRING and INTEGER.
  --> SELECT name FROM students WHERE name > 50;
                                         ^
  HINT: Relational ordering requires compatible types.
```

9. Relational Algebra Intermediate Representation (IR)

Lowering decorated ASTs into decoupled relational algebra operator trees

Relational Operators (`planner/plan.py`)

- `ScanNode` : Base relation access: \(ext{SCAN}(R)\).
- `JoinPlanNode` : Inner join operator: \(R owtie_{ heta} S\).
- `FilterNode` : Relational selection: \(\sigma_{ heta}(R)\).
- `ProjectNode` : Relational projection: \(\pi_{c_1, \dots, c_k}(R)\).
- `AggregatePlanNode` : Statistical aggregation: \(\gamma_{f_1, \dots, f_m}(R)\).
- `SortNode` & `LimitNode` : Ordering \(au_{c}(R)\) and truncation \(\lambda_{k}(R)\).

Generated Logical Plan Tree (IR)

```
\-- LIMIT [3]
    \-- SORT [cgpa DESC]
        \-- PROJECT [name, branch, cgpa]
            \-- FILTER [age >= (18 + 2) AND (1 = 1)]
                \-- SCAN [students]

Formal Relational Algebra Expression:
λ_{3} ( τ_{cgpa DESC} ( π_{name, branch, cgpa} (
    σ_{age ≥ 20} ( SCAN(students) )
) ) )
```

10. Rule-Based Query Optimizer: Constant Folding & Simplification

Compile-time static evaluation and boolean algebraic reductions

Optimization Rule 1: Constant Folding

Recursively pre-computes constant arithmetic and comparisons at compile time:

- \(ext{Fold}((18 + 2)) o 20\)
- \(ext{Fold}(1 = 1) o ext{TRUE}\)
- Eliminates repeated runtime arithmetic inside per-row execution loops.

Optimization Rule 2: Predicate Simplification

Applies classical boolean algebraic axioms:

- **Identity:** \(P \land ext{TRUE} \implies P\)
- **Annihilation:** \(P \land ext{FALSE} \implies ext{FALSE}\)
- **Filter Elimination:** Constant \(ext{TRUE}\) filters pruned entirely.

Optimizer Transformation Report

```
=== QUERY OPTIMIZATION REPORT ===
Rules Applied:
  [x] Constant Folding: folded 2 static expression(s)
  [x] Predicate Simplification: simplified 1 redundant term(s)

--- ORIGINAL PLAN ---
\-- PROJECT [name, branch, cgpa]
    \-- FILTER [age >= (18 + 2) AND (1 = 1)]
        \-- SCAN [students]

--- OPTIMIZED PLAN ---
\-- PROJECT [name, branch, cgpa]
    \-- FILTER [age >= 20]
        \-- SCAN [students]
```

11. Rule-Based Query Optimizer: Algebraic Filter Pushdown

Relocating selection operators past relational joins to minimize intermediate cardinality

Algebraic Equivalence Theorem

If selection predicate \(heta\) references attributes strictly from relation \(R\):

\(\pi(\sigma_{ heta}(R owtie_{\psi} S)) \equiv \pi((\sigma_{ heta}(R)) owtie_{\psi} S)\)

- **Cardinality Reduction:** Filters rows *before* the join rather than after.
- **Join Pruning:** Drops non-qualifying tuples immediately, saving nested-loop comparisons.

Before vs. After Plan Comparison

```
--- ORIGINAL PLAN (FILTER AFTER JOIN) ---
\-- PROJECT [name, dept_name]
    \-- FILTER [students.age > 20]
        \-- JOIN [ON students.branch = departments.dept_name]
            |-- SCAN [students]
            \-- SCAN [departments]

--- OPTIMIZED PLAN (FILTER PUSHED DOWN) ---
\-- PROJECT [name, dept_name]
    \-- JOIN [ON students.branch = departments.dept_name]
        |-- FILTER [students.age > 20]  <-- PUSHED TO SCAN!
        |   \-- SCAN [students]
        \-- SCAN [departments]
```

12. Language Extensions: Multi-Table JOIN & Aggregates

Cross-relation equi-joins and streaming scalar statistical aggregations

Multi-Table Relational INNER JOIN

```
SELECT s.name, d.dept_name, d.building
FROM students s JOIN departments d
ON s.branch = d.dept_name
WHERE s.cgpa >= 8.5;

+--------+-----------+------------------+
| NAME   | DEPT_NAME | BUILDING         |
+--------+-----------+------------------+
| Asha   | CSE       | Technology Tower |
| Rahul  | CSE       | Technology Tower |
| Ananya | CSE       | Technology Tower |
+--------+-----------+------------------+
3 row(s) in set (Scanned: 10, Filtered: 4)
```

Scalar Statistical Aggregations

```
SELECT COUNT(*), AVG(cgpa), MAX(cgpa), MIN(age)
FROM students;

+----------+-----------+-----------+----------+
| COUNT(*) | AVG(CGPA) | MAX(CGPA) | MIN(AGE) |
+----------+-----------+-----------+----------+
| 7        | 8.43      | 9.4       | 19       |
+----------+-----------+-----------+----------+
1 row(s) in set (Scanned: 7, Filtered: 0)

Complexity: Single streaming pass O(N) time,
O(1) auxiliary accumulator memory.
```

13. In-Memory Execution Engine & Performance Benchmarks

Empirical validation of optimization speedups across 100, 1,000, and 10,000 synthetic rows

Empirical Benchmark Results Table

Rows (N)	Unoptimized	Optimized	Speedup	Equivalence
100	4.615 ms	2.442 ms	**1.89x**	100% Match
1,000	48.331 ms	25.017 ms	**1.93x**	100% Match
10,000	504.321 ms	220.880 ms	**2.28x**	100% Match

Complexity Analysis

- **Constant Folding:** Precomputed static arithmetic eliminates 10,000 runtime additions.
- **Filter Pushdown:** Input to join cut from 10k to 2,672 rows, saving 36,640 pairwise checks.

Benchmark Telemetry Output

```
============================================================
QUERYLANG BENCHMARK: UNOPTIMIZED vs. OPTIMIZED IR
============================================================
Query:
  SELECT students.name, departments.dept_name
  FROM students JOIN departments
  ON students.branch = departments.dept_name
  WHERE students.age > (18 + 2) AND students.cgpa > 8.0;

Rules Applied:
  [x] Constant Folding: folded 1 static expression
  [x] Filter Pushdown: pushed filter to SCAN[students]

Observations:
1. Average speedup exceeds 2.0x on large datasets.
2. Verified 100% result equivalence across all runs.
```

14. Formal Quality Assurance: 25-Point Verification Matrix

100% test pass rate across all compiler phases, optimizations, and language extensions

Formal Verification Matrix (T-01 to T-25)

IDs	Category	Verified Feature	Status
T-01 - T-03	Projections	Basic, Multi-column, Relational Filter	**PASS**
T-04 - T-06	Compound	AND, OR, Wildcard (*) Selection	**PASS**
T-07 - T-08	Ordering	ORDER BY ASC/DESC, LIMIT Truncation	**PASS**
T-09 - T-11	Front-End	Lexical Error, Missing Col, Missing FROM	**PASS**
T-12 - T-15	Semantics	Unknown Table, Unknown Col, Type Error	**PASS**
T-16	Pipeline	6-Stage Full Compiler Inspection	**PASS**
T-17 - T-19	Optimizer	Constant Folding, Simplification, Pushdown	**PASS**

Automated Test Runner Output

```
================================================================================
      QUERYLANG COMPILER ENGINE - REVIEW 3 FINAL VERIFICATION MATRIX
================================================================================
Total Verification Matrix Tests: 25 | Passed: 25 | Failed: 0
Verification Status: 100% PASS RATE

Running Formal PyUnit Test Suites...
test_basic_tokenization (test_lexer.TestLexer) ... ok
test_invalid_character_error (test_lexer.TestLexer) ... ok
test_syntax_error_missing_column (test_parser.TestParser) ... ok
test_semantic_error_type_mismatch (test_semantic.TestSemanticAnalyzer) ... ok
test_constant_folding (test_optimizer.TestOptimizerAndExtensions) ... ok
test_filter_pushdown_past_join (test_optimizer.TestOptimizerAndExtensions) ... ok
test_relational_join_execution (test_optimizer.TestOptimizerAndExtensions) ... ok
test_aggregates_execution (test_optimizer.TestOptimizerAndExtensions) ... ok
----------------------------------------------------------------------
Ran 26 tests in 0.011s -- OK
```

15. Conclusion, Viva Defense & Engineering Roadmap

Summary of completed compiler milestones and future research directions

Summary of Achievements

- **Complete 6-Phase Pipeline:** Lexer $ o$ Parser $ o$ Semantic Analyzer $ o$ Planner $ o$ Optimizer $ o$ Engine.
- **Genuine Compiler Implementation:** First-principles Python standard library without external regex or parser generator shortcuts.
- **Algebraic Optimizer:** Proven speedups of up to 2.28x on 10,000 synthetic rows via constant folding and filter pushdown.
- **Deliverables:** 21-Chapter Report, 16-Slide Presentation Deck, interactive CLI, and downloadable PDFs.

Future Engineering Roadmap

- **Cost-Based Optimization (CBO):** Using catalog histogram statistics to choose optimal join orderings.
- **Bytecode Virtual Machine (VDBE):** Lowering relational plans to stack/register bytecode to eliminate interpreter overhead.
- **Persistent Storage Engine:** Implementing a slotted-page buffer pool manager and disk-based B+Tree index.

Thank you! Ready for Viva Questions & Live Demonstration.